

Introduction to Blockchain and Smart Contracts

Lecture 7: Bitcoin Scripts

Subodh Sharma | IIT Delhi | Feb 05, 2026



Bitcoin Scripts

- The *Script* language is small, each instr. takes a byte
 - In all 256 instructions - 15 are currently disabled and 75 reserved
- Basic logic, if-then-else, throw errors, returning early
- Crypto instructions — hash function, CHECKMULTISIG
 - Multisign check requires t out of n pub keys to be valid
- How does the stack machine work?
 - Two types: Opcodes and Data instructions; put data instructions on the stack when opcode is seen, data from top of the stack is popped.

```
<sig>  
<pubKey>  
-----  
OP_DUP  
OP_HASH160  
<pubKeyHash?>  
OP_EQUALVERIFY  
OP_CHECKSIG
```

Bitcoin Scripts

- The *Script* language is small, each instr. takes a byte
 - In all 256 instructions - 15 are currently disabled and 75 reserved
- Basic logic, if-then-else, throw errors, returning early
- Crypto instructions — hash function, CHECKMULTISIG
 - Multisign check requires t out of n pub keys to be valid
- How does the stack machine work?
 - Two types: Opcodes and Data instructions; put data instructions on the stack when opcode is seen, data from top of the stack is popped.

```
<sig>  
<pubKey>  
-----  
OP_DUP  
OP_HASH160  
<pubKeyHash?>  
OP_EQUALVERIFY  
OP_CHECKSIG
```


Bitcoin Scripts - MULTISIG Addresses

- Suppose A buys a service from B. But B maintains a MULTISIG Address. How does A make the payment?
 - B asks A to send coins to a hash of a *script* (instead of a public address)
 - A condition is imposed that the script is revealed that has a given hash, along with the data that makes the script evaluate to *true*
 - This is *pay-to-script-hash* transaction type
- Trivia about P2SH!



Bitcoin Scripts - MULTISIG Addresses

- The redeem script `OP_2 PubA PubB PubC OP_3 OP_CHECKMULTISIG`
- Hash the script `HASH160(redeemScript) = RIPEMD160(SHA256(redeemScript))`
- Output locking script (scriptPubKey): `OP_HASH160 <scriptHash> OP_EQUAL`
- The spender provides 2 signatures and the redeem script:
`OP_0 sig1 sig2 <redeemScript>`
- Script execution: hash the redeem script and check again scriptHash ...

Applications of Bitcoin Scripts

- **Escrow Transactions**
 - Partially addresses the **fair exchange problem**
 - Actually **smart contracts** with **atomic swaps** fully solves this problem!
 - Escrows can be implemented using MULTISIG.
 - A has to send coins to B, J is an arbitrator; A creates a 2-of-3 MULTISIG txn
 - The txn is on Blockchain and the coins are held in an escrow between A, B and J s.t. any two can specify where the coins should go.

Green Addresses

- A wants to pay B, but for whatever reason B is either offline or can't access the BC
- A can send to a green address (typically controlled by a Bank) that B trusts and the Bank pays to Bob through a transaction
- The Bank has to be trusted not to double-spend

Other Applications

- **Efficient Micropayments**
 - HINT: Using MULTISIG
- **Lock time:** To avoid situations where money sits in Escrow forever

Hash Locks & Time Locks

Can spend the funds only if know a secret s

- Pick a hash function H
- Secret: s
- Public hash: $h = H(s)$
- **Hashlock:** One can spend the funds only if they reveal a secret s s.t.
 $H(s) = h$
- **Timelock:** One can spend the funds only after a certain time/block height
 - Two types: **Absolute** and **Relative**

HTLC: Hashlock + Timelock

- Either you **reveal the secret before deadline** and claim funds or **after deadline the original funder can refund**
- Main use cases: HTLCs act as the engine for multi-hop payments in the Bitcoin world (lightning network)

```
IF
    # Success path
    <receiver_pubkey> CHECKSIGVERIFY
    SHA256 <h> EQUALVERIFY          # receiver
ELSE
    # Refund path
    <timeout> CHECKLOCKTIMEVERIFY DROP
    <sender_pubkey> CHECKSIGVERIFY
ENDIF
```

Atomic Swaps

- Exchange of two assets where **either both transfers happen or both parties can refund**
- Atomic swaps can be on the same chain or across chains, as well
 - BTC $\longleftrightarrow$ LTC
- **A** has 1 BTC and **B** has 10 LTC and they want to swap 1 BTC $\longleftrightarrow$ 10 LTC
 - Agree on a hash function (say SHA 256) ; timeout **T1** on Bitcoin and **T2** on Litecoin
 - **T1 > T2:**

Atomic Swaps

- **A** generates the secret s and then $h = \text{SHA256}(s)$
- **A** shares h with **B**
- **A** locks 1 BTC in an HTLC_BTC (on Bitcoin chain)
 - **B** can claim it if he provides his signature, the secret s s.t. $H(s) = h$ and before $T1$
- Similarly **B** verifies HTLC_BTC and locks 100 LTC in HTLC_LTC
- **A** claims **B's** LTC by revealing **s before $T2$**
- Now **B** extracts **s** , signs it and spends HTLC_BTC before $T1$ (that is why $T1 > T2$)
- The swap is now complete

HTLCs with Atomic Swaps and the Fair Exchange Problem

- **B** can't get BTC until **A** has already revealed the secret **s**
- **A** can't get **B's** LTC until **s** is revealed
- The solution guarantees fairness but **not liveness**
 - Some one can still waste your time by locking and aborting

Storage: Hot and Cold